PUKYONG NATIONAL UNIVERSITY

국립부경대학교 공과대학

시스템경영·안전공학부

소프트웨어공학 4주차 강의 교재

AI로 Git 다루기

팀 풀 리퀘스트를 Claude Code로 병합하기

2026학년도 2학기 · 2026년 9월 22일(화) 온라인 강의 1부

최성철 교수

이 교재는 9월 22일 온라인 수업 1부(약 32분)의 녹화와 실습 저장소의 실제 기록을 바탕으로 엮었습니다. 충돌과 동기화를 보여 주는 터미널 출력은 같은 상황을 내 컴퓨터에서 재현해 Git 2.50으로 실행한 결과입니다.

이 교재를 읽는 법

이 교재는 소프트웨어공학 4주차 강의(2026년 9월 22일, Zoom 온라인) 가운데 1부를 책처럼 다시 엮은 것입니다. 1부에서는 3주차 실습과제로 각 팀이 보낸 풀 리퀘스트(PR)를 교수가 Claude Code로 받아 병합하는 과정을 처음부터 끝까지 보여 주었습니다. 2부의 [4] 기획 강의는 따로 다룹니다.

3주차 교재 [강의노트 03] Git과 GitHub가 개념을 하나씩 쌓았다면, 이 교재는 그 개념을 AI에게 일을 시키는 데 어떻게 쓰는지를 보여 줍니다. 수업에서 교수는 `git` 명령을 거의 치지 않았습니다. 명령은 AI가 쳤고, 교수는 무엇을 할지 말하고, 선택지 가운데 하나를 고르고, 결과가 맞는지 확인했습니다. 이 교재도 그 순서를 따라갑니다.

수업에서 거듭 한 말이 있습니다. "AI가 하라는 대로 하면 되는 거 아니에요?"라고 생각하기 쉽지만, 지금 무슨 일이 일어나는지 모르면 AI가 한 일이 맞는지 틀린지 가릴 수 없습니다. PR, CLI, push, upstream 같은 말을 알아야 하는 이유입니다. 개념이 흐릿하면 강의노트 03의 해당 장을 다시 보세요. 이 교재 곳곳에 어느 장을 보면 되는지 적어 두었습니다.

상자	쓰임
핵심	꼭 기억해야 할 개념을 한두 문장으로 정리합니다.
수업에서	수업 중에 나온 이야기와 문답을 옮깁니다.
AI에게 이렇게 시킨다	수업에서 Claude Code에 실제로 한 요청을 다듬어 옮깁니다.
주의	실수하면 되돌리기 어려운 부분입니다.
쉬어가기	본문과 조금 떨어진 여담입니다.

터미널 상자에서 `$`로 시작하는 굵은 줄은 입력한 명령이고, 나머지는 돌려받은 출력입니다. 수업에서는 이 명령을 Claude Code가 대신 실행했습니다. 여러분이 직접 칠 필요는 없지만, AI의 작업 기록에 이런 줄이 지나갈 때 무슨 뜻인지 읽을 수 있어야 합니다.

차례

1 명령은 AI가 치고, 사람은 판단한다 1

1.1 오늘 수업에서 한 일 1

1.2 사람의 일과 AI의 일 2

2 작업 공간 만들기 3

2.1 Code 탭과 작업 폴더 3

2.2 “가져와” 라고만 해도 clone한다 3

3 gh: GitHub을 터미널에서 5

3.1 “PR한 걸 gh로 머지하자”는 무슨 뜻인가 5

3.2 설치: 안 되면 새 터미널, 그래도 안 되면 다시 묻기 6

3.3 로그인: device code 방식 7

4 PR 확인과 병합을 AI에게 8

4.1 “네가 직접 확인해 줘” 8

4.2 실습 저장소의 구조 9

4.3 첫 PR은 그냥 합쳐졌다 9

5 충돌을 AI와 함께 풀기 10

5.1 충돌은 왜 생겼나 10

5.2 “내가 의사결정할 수 있게 해 줘” 11

5.3 네 가지 선택지 12

5.4 B를 실제로 어떻게 했나 13

6 기다리지 말고 동시에 15

6.1 병렬성 15

6.2 결과는 눈으로 확인한다 16
6.3 웹 화면도 AI가 조작한다 16

7 마무리: README 갱신과 upstream 동기화 17
7.1 commit은 저장, push는 원격으로 17
7.2 가장 최신 버전은 누가 가지고 있나 18
7.3 동기화를 재현해 보면 18
7.4 2차 실습 19

A 수업에서 쓴 요청 모음 20

B gh 명령 한눈에 21

C 헷갈리는 짝 22

D 복습 문제 23

제 1 장

명령은 AI가 치고, 사람은 판단한다

1.1 오늘 수업에서 한 일

3주차 실습과제는 팀 풀 리퀘스트였습니다. 각 팀은 교수 저장소 `TEAMLAB-Lecture/swe-2026-git-practice`를 자기 계정으로 fork하고, 팀원끼리 PR을 주고받은 뒤, 마지막에 팀 저장소에서 교수 저장소로 PR을 열었습니다. 9월 21일 마감까지 네 팀의 PR이 모두 들어왔습니다.

9월 22일 수업 1부는 이 PR 네 개를 교수가 받아 합치는 과정이었습니다. 순서는 그림 1.1과 같습니다.

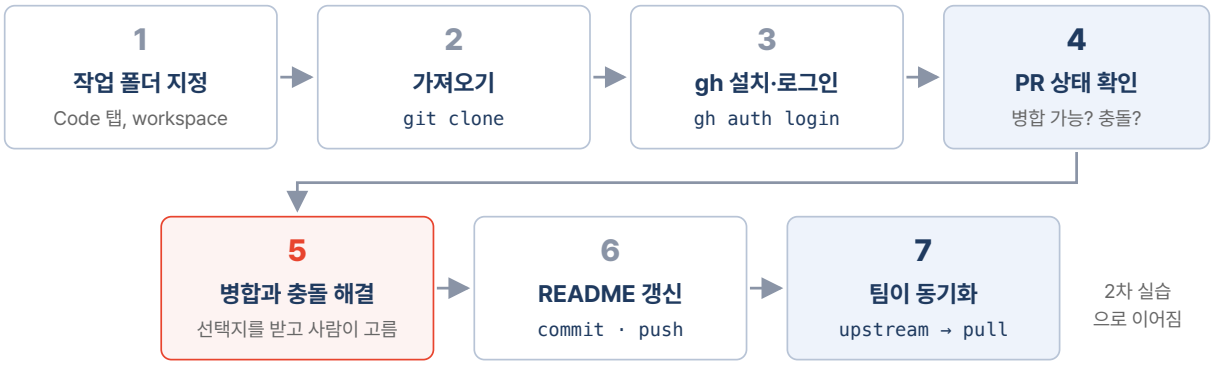


그림 1.1 9월 22일 수업 1부의 흐름. 5단계에서 충돌이 나서 사람이 결정했다

1. Claude 데스크톱 앱의 Code 탭에서 작업 폴더를 정합니다.
2. 교수 저장소를 내 컴퓨터로 가져옵니다(clone).
3. GitHub을 터미널에서 다루는 도구 `gh`를 설치하고 로그인합니다.
4. 들어온 PR 네 개의 상태를 확인합니다. 합칠 수 있는지, 충돌이 있는지 봅니다.
5. 병합합니다. 충돌이 난 PR은 AI에게 선택지를 받아 사람이 고릅니다.
6. README를 지금 상태에 맞게 고치고 commit, push합니다.
7. 각 팀이 교수 저장소의 최신 상태를 받아 오도록(upstream 동기화) 2차 실습을 안내합니다.

이 일곱 단계에서 교수가 직접 친 `git` 명령은 거의 없습니다. 그렇다고 교수가 한 일이 없는 것은 아닙니다. 무엇을 할지 정했고, 충돌을 어떻게 풀지 골랐고, AI가 한 일을 VS Code와 GitHub에서 열어 확인했습니다.

1.2 사람의 일과 AI의 일

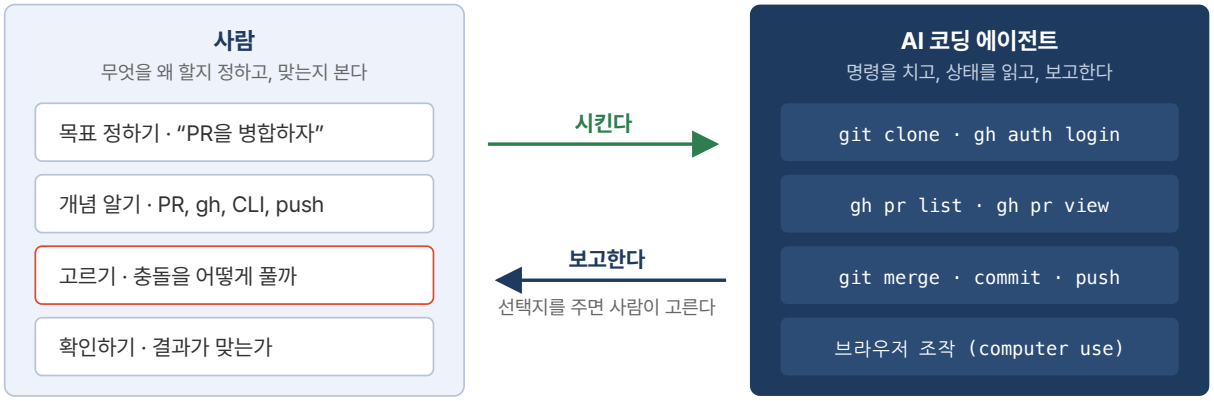


그림 1.2 명령을 치는 일은 AI에게 넘어갔다. 사람에게는 정하고, 고르고, 확인하는 일이 남는다

수업에서 교수는 이렇게 말했습니다. `git status` 같은 명령을 하나하나 치는 일은 이제 여러분이 할 필요가 별로 없고, 대부분 AI가 직접 하는 시대가 이미 왔다고요. 그 대신 사람에게 남는 일이 있습니다.

- **목표를 정합니다.** "들어온 PR을 합치자" 처럼 무엇을 할지는 사람이 말합니다.
- **개념을 압니다.** PR, CLI, push, upstream 같은 말을 알아야 AI에게 시킬 수 있고, AI의 보고를 알아들을 수 있습니다.
- **고릅니다.** 충돌을 어떻게 풀지처럼 정답이 하나가 아닌 문제는 AI가 선택지를 내고 사람이 고릅니다.
- **확인합니다.** AI가 "다 했다" 고 해도 결과를 열어 봅니다.

> **핵심 · 검증할 수 있어야 시킬 수 있다**
>
> AI가 하라는 대로만 따라가면, AI가 틀렸을 때 알아챌 방법이 없습니다. 명령어는 외우지 않아도 되지만, 지금 무슨 일이 일어나는지는 알아야 합니다.

> **수업에서**
>
> 화면을 따라가던 학생에게 교수가 "내가 지금 뭐 하는 거지?" 라고 물었습니다. 첫 학생은 "터미널에 있는 코드를 GitHub에 연동시키는 과정 같다" 고 답했지만 왜 해야 하는지는 모르겠다고 했습니다. 다음 학생은 "아까 그 PR을 머지하려고" 라고 답했습니다. 교수는 "이런 용어에 익숙하지 않으면 평생 못 따라간다" 고 하면서, 다음 시험에서 이런 개념을 묻겠다고 예고했습니다. 이 교재 3장에서 그 답을 정리합니다.

제 2 장

작업 공간 만들기

2.1 Code 탭과 작업 폴더

수업에서는 Claude 데스크톱 앱을 썼습니다. 앱 위쪽에는 채팅, 코워크, 코드 탭이 있는데, 프로그램을 다루는 일은 항상 **Code 탭**에서 합니다. Code 탭에서 새 세션을 만들면 작업할 폴더를 정할 수 있습니다. 이 폴더가 AI가 파일을 읽고 쓰고 명령을 실행하는 자리입니다.

교수는 처음에 연구실 폴더를 지정했다가 `D:\workspace`로 바꿨습니다. 코드를 다룰 때는 동기화 드라이브(구글 드라이브, OneDrive)보다 이런 작업용 폴더가 편합니다. 동기화 프로그램이 Git 저장소 안의 수많은 작은 파일을 계속 올리고 내리면 느려지고, 가끔 파일이 꼬이기도 합니다.

Claude Code만 되는 것은 아닙니다. Codex나 Gemini CLI 같은 다른 코딩 에이전트도 이번 수업의 흐름대로 동작합니다. 수업에서는 윈도우에서 쓰기 편한 Claude를 권했습니다.

2.2 "가져와"라고만 해도 clone한다

작업 폴더를 정한 뒤, 교수는 GitHub의 교수 저장소를 내 컴퓨터로 가져오라고 시켰습니다. 원격 저장소를 통째로 복사해 오는 일을 Git에서는 **clone**이라고 합니다(강의노트 03 5.2절).

> **AI에게 이렇게 시킨다**
>
> > GitHub에 있는 swe-2026-git-practice 저장소를 가져올 거야. 현재 폴더 기준으로 어디에 두면 좋을지 먼저 생각해 보자.
>
> > 좋아. Lectures 폴더를 만들고 거기에 클론하자.

AI는 먼저 작업 폴더의 구조를 살펴보고, 저장소를 어디에 둘지 물었습니다. 교수가 Lectures 폴더를 만들라고 하자, 폴더를 만들고 clone을 끝냈습니다. 결과는 `D:\workspace\Lectures\swe-2026-git-practice`입니다.

교수는 이렇게 덧붙였습니다. "클론"이라는 말을 몰라도 "가져와"라고 하면 AI가 알아서 clone합니다. 그래도 그때 어떤 일이 일어나는지는 여러분이 알아야 합니다. clone은 파일만 복사하는 것이 아니라 커밋 기록 전체와 원격 저장소 주소(`origin`)까지 함께 가져옵니다. 그래서 가져온 뒤에 바로 pull과 push를 할 수 있습니다.

쉬어가기 · 요금제 이야기

수업 중에 AI 도구 요금제 이야기가 나왔습니다. 교수는 월 3만 원대 요금제로는 이런 작업을 거의 못 하니 10만 원대 요금제를 권했습니다. 오늘 같은 작업은 가벼운 모델(Sonnet)로도 충분하다고 했습니다. 교수 자신은 30만 원대 요금제를 쓰는데, 한 주 사용량을 이틀 만에 거의 다 쓴다고 합니다. 그만큼 일을 AI에게 넘기고 있고, 체감으로는 예전보다 서른 배쯤 일을 더 하는 것 같다고 했습니다.

제 3 장

gh: GitHub을 터미널에서

3.1 "PR한 걸 gh로 머지하자"는 무슨 뜻인가

저장소를 가져온 뒤 교수는 AI에게 이렇게 시켰습니다.

> **AI에게 이렇게 시킨다**
>
> > 포크해서 PR한 걸 gh를 사용해서 머지할 거야. 어떻게 할지 고민해 보자.

이 한 문장에 이번 주의 핵심 용어가 다 들어 있습니다. 수업에서 교수가 학생에게 뜻을 물었고, 학생은 "수정 요청한 것을 CLI에서 합치는 명령을 실행하겠다는 뜻"이라고 답했습니다. 하나씩 풀면 이렇습니다.

말	뜻
fork	남의 GitHub 저장소를 내 GitHub 계정으로 복사하는 것. 각 팀이 교수 저장소를 fork해서 팀 저장소를 만들었습니다.
PR (풀 리퀘스트)	"내가 고친 것을 가져가(pull) 주세요"라는 요청. 팀 저장소에서 교수 저장소로 열었습니다.
gh	GitHub이 만든 명령줄 도구(GitHub CLI). PR 보기, 병합, 댓글 달기처럼 GitHub 웹에서 하던 일을 터미널에서 합니다.
CLI	Command Line Interface. 버튼과 창(GUI) 대신 글자로 명령을 주고받는 방식입니다.
머지 (merge)	두 갈래의 작업을 하나로 합치는 것. 여기서는 PR의 변경을 교수 저장소의 main에 합칩니다.

> **핵심 · git과 gh**
>
> `git`은 저장소 안의 기록(커밋, 브랜치, 병합)을 다루고, `gh`는 GitHub이라는 서비스 위의 일(PR, 이슈, 리뷰 댓글, fork)을 다룹니다. PR은 Git이 아니라 GitHub의 기능이라서 PR을 다루려면 `gh`가 필요합니다.

그렇다면 AI는 왜 `gh`를 쓰려고 할까요? PR은 GitHub 서버에 있습니다. AI가 PR 목록을 보고, 충돌이 있는지 확인하고, 병합 버튼을 대신 누르려면 GitHub에 말을 걸 수단이 필요합니다. 웹 브라우저로 할 수도 있지만, `gh`를 쓰면 AI가 명령 한 줄로 정확한 결과를 받아 볼 수 있습니다. AI에게는 버튼보다 글자가 다루기 쉽습니다.

3.2 설치: 안 되면 새 터미널, 그래도 안 되면 다시 묻기

교수 컴퓨터에는 `gh`가 없었습니다. AI는 `gh`가 설치되어 있지 않다고 알리고, 설치 명령을 제안했습니다. 윈도우라면 보통 다음과 같습니다.

```
$ winget install --id GitHub.cli
```

맥이라면 `brew install gh`입니다. 설치 명령은 AI가 실행해도 되고, AI가 보여 준 명령을 복사해 직접 터미널에 붙여 넣어도 됩니다. 교수는 과정을 보여 주려고 Code 탭 아래쪽에 터미널을 열어 직접 실행했습니다.

설치가 끝났는데도 터미널이 `gh`를 찾지 못했습니다. 이럴 때 교수가 한 일은 단순합니다. 터미널에 나온 오류를 그대로 복사해서 AI에게 붙여 넣었습니다. AI의 답은 "새 터미널을 열어야 인식한다" 였습니다.

이유는 **PATH**에 있습니다. 터미널은 명령을 찾을 폴더 목록(PATH)을 처음 열릴 때 한 번 읽어 둡니다. 프로그램을 새로 설치하면 PATH에 폴더가 추가되지만, 이미 열려 있던 터미널은 예전 목록을 그대로 씁니다. 그래서 터미널을 닫고 새로 열어야 합니다. 새로 열었는데도 안 되면, 다시 그 상황을 AI에게 알리고 AI가 시키는 대로 다음 명령을 붙여 넣으면 됩니다.

> **AI에게 이렇게 시킨다 · 막혔을 때**
>
> > (터미널 오류를 그대로 붙여 넣고) 이렇게 나와. 왜 안 되는 거야?
> > 새 터미널 열었는데도 안 돼.

3.3 로그인: device code 방식

gh를 설치했으면 로그인해야 합니다. gh는 내 컴퓨터의 프로그램일 뿐이라, 누구의 GitHub 계정으로 일하는지 알려 주어야 PR을 보고 병합할 수 있습니다. 수업에서 학생에게 이것을 왜 하는지 물었을 때 교수가 정리한 답은 이렇습니다. 내 컴퓨터에 깔린 GitHub CLI가 내 GitHub 계정에 로그인된 상태로 연결되게 하려는 것입니다.

```
$ gh auth login
```



설치 직후 "gh를 찾을 수 없음"이 나오면 새 터미널을 연다. 이미 열려 있던 터미널은 새 프로그램 경로(PATH)를 모른다.

그림 3.1 gh의 로그인. 터미널이 보여 준 일회용 코드를 브라우저에 넣고 승인하면, 토큰이 내 컴퓨터에 저장된다

gh auth login을 실행하면 터미널에 ABCD-1234 같은 일회용 코드가 나오고, 브라우저가 github.com/login/device 페이지를 엽니다(그림 3.1). 코드를 넣고 내 계정으로 로그인해 권한을 승인하면, GitHub이 gh에게 **토큰**을 줍니다. 토큰은 비밀번호 대신 쓰는 출입증 같은 것으로, 내 컴퓨터의 보안 저장소(윈도우 자격 증명 관리자, 맥 키체인)에 저장됩니다. 그 뒤로는 이 컴퓨터의 gh가 내 이름으로 GitHub 일을 할 수 있습니다.

로그인이 됐는지는 gh auth status로 확인합니다.

> **주의 · 토큰은 비밀번호와 같다**
>
> 토큰이 있으면 누구든 내 이름으로 저장소를 고치고 지울 수 있습니다. 화면에 나온 토큰을 채팅이나 과제에 붙여 넣거나, 코드 파일에 적어 커밋하지 마세요(강의노트 03 6.8절). 일회용 코드(ABCD-1234)는 몇 분 뒤 쓸모가 없어지지만, 토큰은 취소하기 전까지 살아 있습니다.

제 4 장

PR 확인과 병합을 AI에게

4.1 “네가 직접 확인해 줘”

로그인을 마치자 AI는 PR 상태를 확인하는 명령을 알려 주면서 실행해 보라고 했습니다. 교수는 직접 치지 않고 이렇게 답했습니다.

> **AI에게 이렇게 시킨다**
>
> > 내가 로그인했어. 네가 직접 확인해 줘.

교수는 평소에도 이렇게 한다고 했습니다. 확인하는 명령을 AI가 알려 주면, 사람이 치지 않고 AI에게 직접 하라고 시킵니다. 명령을 치는 것은 AI가 더 빠르고 정확합니다. 사람은 AI가 가져온 결과를 읽으면 됩니다.

이때 AI가 내부에서 실행하는 명령은 대략 다음과 같습니다. 여러분이 칠 필요는 없지만, 작업 기록에 지나가면 알아볼 수 있어야 합니다.

```
$ gh pr list --repo TEAMLAB-Lecture/swe-2026-git-practice
$ gh pr view 3 --json mergeable,mergeStateStatus,files
$ gh pr checks 3
```

명령	알아내는 것
`gh pr list`	열려 있는 PR 목록. 번호, 제목, 보낸 브랜치
`gh pr view <번호>`	PR 하나의 설명, 바뀐 파일, 병합 가능 여부
`gh pr checks <번호>`	자동 검사(GitHub Actions) 결과
`gh pr diff <번호>`	PR이 바꾼 내용(diff)

수업 화면에서 AI는 PR마다 병합 가능 여부와 충돌 여부, 자동 검사 결과를 표로 정리해 보여 주었습니다. 이 저장소에는 자동 검사가 설정되어 있지 않아서 검사 칸은 모두 “없음”이었습니다.

4.2 실습 저장소의 구조

PR이 어디서 어디로 가는지 그림으로 보면 그림 4.1와 같습니다. 교수 저장소가 가운데 있고, 네 팀이 각자 fork한 저장소에서 PR을 보냈습니다. 교수 컴퓨터는 교수 저장소를 clone해 두었고, Claude Code는 그 폴더에서 `gh`와 `git`을 실행합니다.

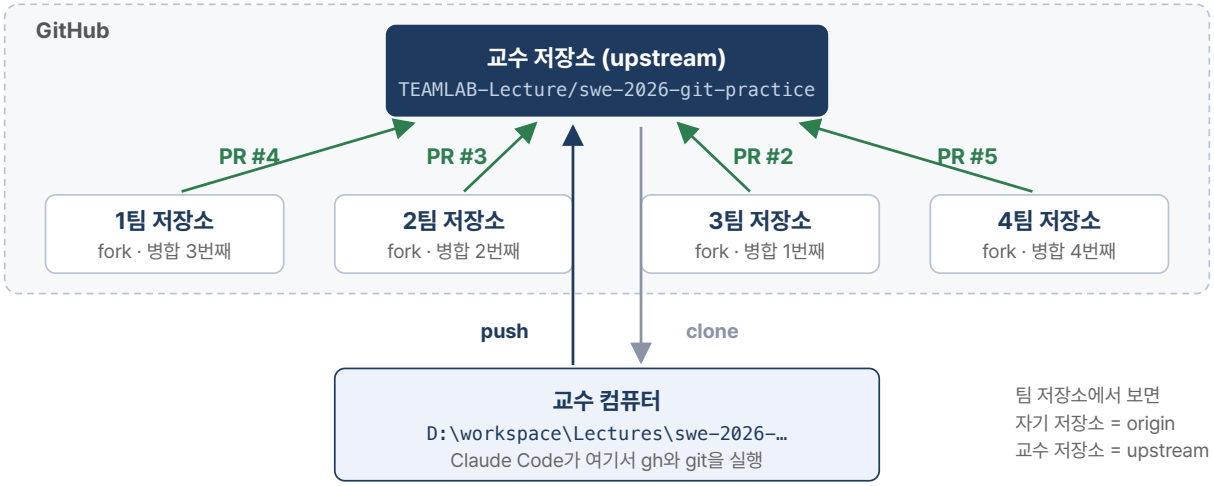


그림 4.1 3주차 실습의 저장소 구조와 실제 병합 순서. 팀 저장소 입장에서 교수 저장소는 upstream이다

실제 기록을 보면 PR #2(3팀)가 먼저 병합되었고, 이어서 #3(2팀), #4(1팀), #5(4팀) 순으로 병합되었습니다. 병합은 9월 22일 13시 58분부터 14시 9분 사이에 끝났습니다. 네 팀의 PR을 합치는 데 11분이 걸렸습니다.

4.3 첫 PR은 그냥 합쳐졌다

3팀의 PR은 아무 문제 없이 합쳐졌습니다. 교수 저장소의 main이 과제를 시작할 때 그대로였기 때문입니다. main에 새 커밋이 없으니 Git은 3팀의 커밋을 main 뒤에 그대로 이어 붙이면 됩니다.

문제는 그다음이었습니다. 3팀의 PR이 병합되어 main이 바뀌자, 나머지 세 팀의 PR은 모두 충돌 상태가 되었습니다. 수업 화면에서는 AI가 "2번(2팀 PR)은 병합하려 했더니 충돌한다"고 보고했습니다.

제 5 장

충돌을 AI와 함께 풀기

5.1 충돌은 왜 생겼나

교수가 학생에게 물었습니다. 충돌이 왜 생긴 걸까? 학생은 다른 팀의 PR 때문인 것 같다고 답했고, 교수는 이렇게 정리했습니다. 모든 팀이 교수 저장소의 같은 시점에서 갈라져 나가 각자 고쳤는데, 각 팀이 같은 곳을 서로 다르게 고쳤기 때문입니다.

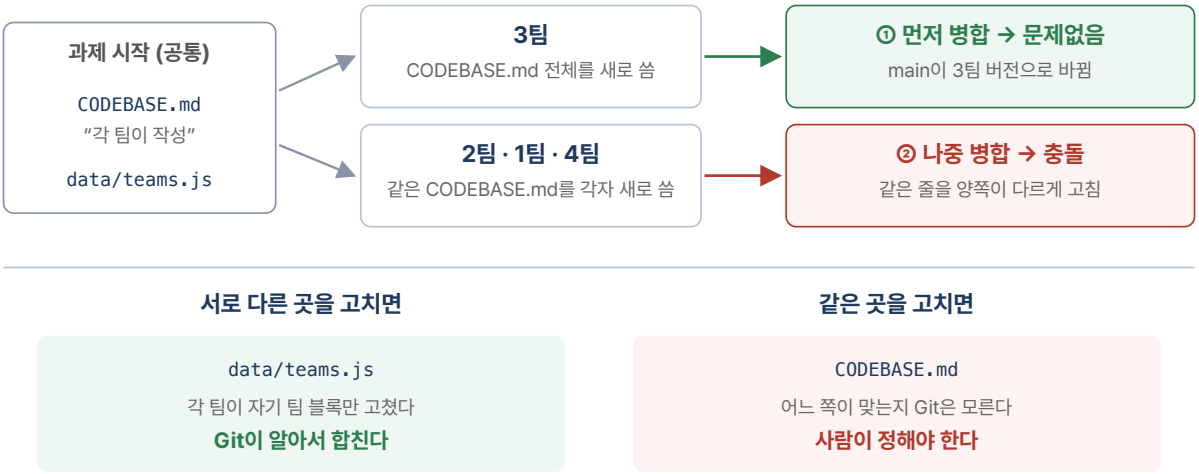


그림 5.1 네 팀이 같은 CODEBASE.md를 각자 새로 썼다. 먼저 병합된 3팀 뒤로 나머지는 모두 충돌했다

과제에서 각 팀이 고친 파일은 두 개였습니다.

- `data/teams.js`: 팀과 팀원 목록입니다. 과제 안내에 "각 팀은 자기 팀 블록만 고친다"고 적어 두었습니다. 팀마다 다른 줄을 고쳤으니 Git이 알아서 합쳤습니다.
- `CODEBASE.md`: 코드베이스 설명 문서입니다. 과제를 시작할 때는 비어 있었고, 네 팀이 모두 이 파일의 같은 자리를 자기 글로 채웠습니다. 먼저 병합된 3팀의 글과 나중 팀의 글이 같은 줄에서 부딪쳤습니다.

> **핵심 · 충돌은 Git이 "모르겠다"고 하는 것**
>
> 두 사람이 서로 다른 곳을 고치면 Git이 알아서 합칩니다. 같은 곳을 서로 다르게 고치면 Git은 어느 쪽이 맞는지 알 수 없어서 멈춥니다. 이것이 충돌입니다. 충돌은 오류가 아니라, 사람이 정해 달라는 요청입니다(강의노트 03 6.6절).

같은 상황을 내 컴퓨터에서 재현하면 다음과 같습니다. 교수 저장소에 3팀의 작업을 먼저 합치고, 이어서 2팀의 작업을 합치려는 장면입니다.

```
$ git merge --no-edit team3/main
Updating 117e959..f605f25
Fast-forward
 CODEBASE.md | 3 ++-
 1 file changed, 2 insertions(+), 1 deletion(-)
$ git merge --no-edit team2/main
Auto-merging CODEBASE.md
CONFLICT (content): Merge conflict in CODEBASE.md
Automatic merge failed; fix conflicts and then commit the result.
$ git status --short
UU CODEBASE.md
```

3팀은 `Fast-forward`로 그냥 이어 붙었습니다. 2팀을 합치려 하자 `CONFLICT`가 났고, `git status`에는 양쪽이 모두 고친 파일이라는 뜻의 `UU`가 붙었습니다. 파일을 열어 보면 Git이 두 버전을 나란히 적어 두었습니다.

```
$ cat CODEBASE.md
# CODEBASE

<<<<<<< HEAD
- index.html: 화면 구조
- app.js: 카드 그리기와 검색
=======
이 프로젝트는 팀 카드를 보여 주는 웹페이지다.
- data/teams.js: 팀 정보
>>>>>>> team2/main
```

`<<<<<<< HEAD`부터 `=======`까지가 지금 main에 있는 3팀의 글이고, 그 아래부터 `>>>>>>>`까지가 2팀의 글입니다. 둘 중 하나를 고르든, 둘을 섞든, 사람이 정해서 표시를 지우고 커밋해야 병합이 끝납니다.

5.2 "내가 의사결정할 수 있게 해 줘"

여기서 교수는 AI에게 충돌을 알아서 풀라고 하지 않았습니다. 이렇게 시켰습니다.

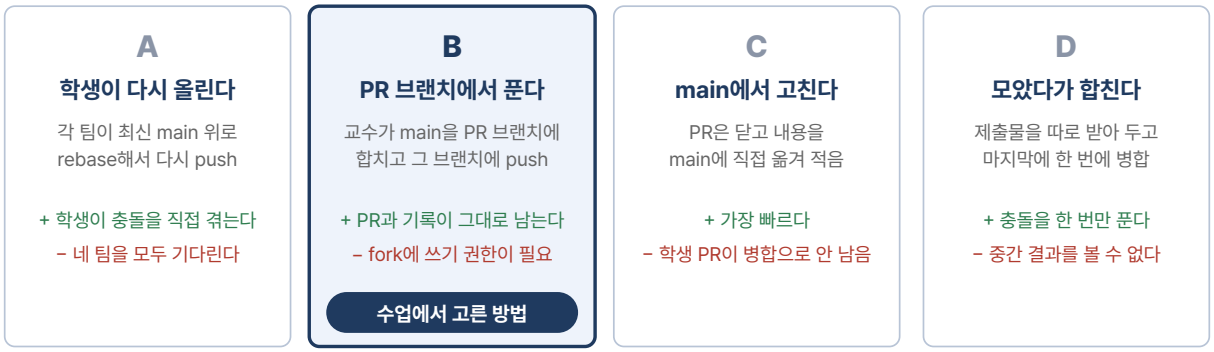
> **AI에게 이렇게 시킨다**
>
> > 충돌의 문제점들을 코드 단위로 나에게 설명해 주고, 내가 어떻게 할지 의사결정할 수 있게 해 줘.

교수는 이 요청이 오늘 수업에서 아주 중요하다고 했습니다. 컴퓨터에게 무조건 다 시키면 안 됩니다. 사실 무조건 시켜도 생각보다 잘합니다. 이번 충돌도 AI에게 맡겼다면 별다른 문제 없이 풀었을 것입니다. 그래도 확인하는 과정은 늘 거쳐야 합니다. 특히 충돌은 누구의 작업을 남기고 누구의 작업을 버릴지 정하는 일이라서, 그 결정은 사람이 해야 합니다.

> **핵심 · 결정할 거리를 달라고 시킨다**
>
> AI에게 "해결해 줘"가 아니라 "내가 결정할 수 있게 선택지와 장단점을 줘"라고 시키세요. 같은 AI라도 무엇을 달라고 하느냐에 따라 사람이 일의 주인으로 남기도 하고, 결과만 받아 드는 사람이 되기도 합니다.

5.3 네 가지 선택지

AI는 충돌한 파일을 코드 단위로 보여 주고, 풀 방법을 몇 가지 내놓았습니다. 수업 화면에 나온 선택지를 정리하면 그림 5.2과 같습니다.



AI가 내놓은 선택지를 정리한 것이다. AI는 고르지 않고, 코드 단위 설명과 장단점까지만 준다.

그림 5.2 충돌을 푸는 네 가지 방법. 수업에서는 B를 골랐다

A. 학생에게 다시 올리라고 한다

각 팀이 교수 저장소의 최신 main을 받아 자기 작업을 그 위로 rebase하고, 충돌을 풀어 다시 push합니다. 학생이 충돌 해결을 직접 겪는다는 장점이 있지만, 네 팀이 모두 고칠 때까지 기다려야 합니다.

B. 교수가 PR 브랜치에서 푼다

교수가 main을 PR 브랜치에 합쳐 충돌을 풀고, 그 결과를 학생 fork의 PR 브랜치에 push합니다. PR에 해결 커밋이 붙으면 PR을 그대로 병합할 수 있습니다. 학생의 PR과 커밋 기록이 그대로 남습니다.

C. main에서 직접 고친다

PR은 닫고, 학생이 쓴 내용을 교수가 main에 직접 옮겨 적습니다. 가장 빠르지만 학생의 PR이 병합으로 남지 않습니다.

D. 모았다가 한 번에 합친다

학생들이 보낸 것을 따로 저장해 두었다가 마지막에 교수가 한꺼번에 합칩니다. 교수는 평소 혼자 일할 때 이 방법을 쓴다고 했습니다. 충돌을 한 번만 풀면 되지만, 중간 결과를 볼 수 없습니다.

교수는 학생에게 넷 가운데 무엇이 가장 나은지 물었고, 학생은 B를 골랐습니다. 수업에서도 B로 진행했습니다.

> **수업에서**
>
> 교수는 여기에 한 가지를 덧붙였습니다. 혼자 일할 때는 이런 관리가 별로 필요 없습니다. 같이 일할 때는 이렇게 충돌이 자주 생기기 때문에, 충돌을 어떻게 풀지는 항상 물어보고 해야 합니다. 여러분이 AI에게 일을 시킬 때도 마찬가지입니다. 남의 작업이 걸린 결정은 AI가 혼자 내리게 두지 마세요.

5.4 B를 실제로 어떻게 했나

실습 저장소의 기록을 보면 B가 어떻게 실행되었는지 알 수 있습니다(그림 5.3).

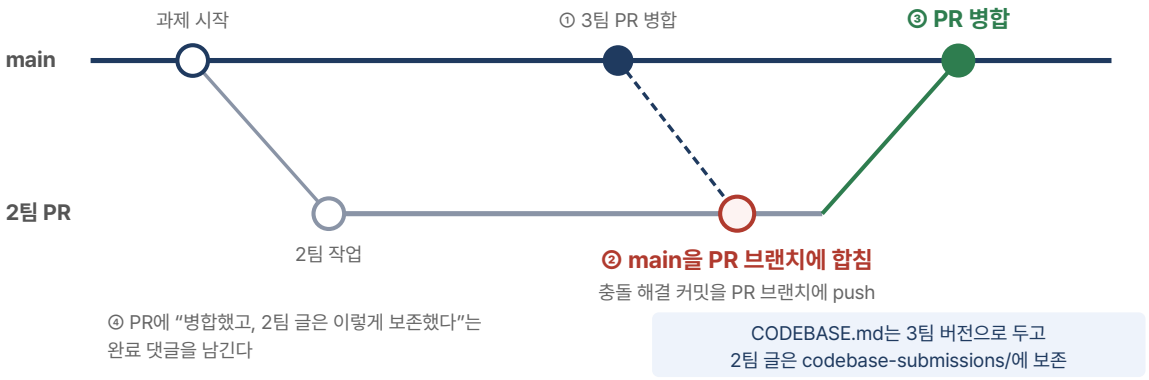


그림 5.3 2팀 PR을 병합한 과정. 충돌 해결 커밋을 PR 브랜치에 먼저 넣고 PR을 병합했다

1. AI가 교수 저장소의 main(3팀 병합 후)을 2팀의 PR 브랜치에 합쳤습니다. 이때 `CODEBASE.md`에서 충돌이 났습니다.
2. 충돌은 이렇게 풀었습니다. 저장소의 공식 `CODEBASE.md`는 먼저 병합된 3팀 버전으로 두고, 2팀이 쓴 글은 지우지 않고 `codebase-submissions/2팀-CODEBASE.md`로 옮겨 보존했습니다.
3. 이 해결 커밋을 2팀 fork의 PR 브랜치에 push했습니다. PR이 다시 "병합 가능"이 되었고, 그대로 병합했습니다.
4. 1팀과 4팀도 같은 방법으로 처리했습니다. 병합이 끝난 PR마다 무엇을 어떻게 합쳤는지 댓글을 남겼습니다.

재현 저장소에서 같은 해결을 하면 다음과 같습니다. 3팀 버전을 남기고(`--ours`), 2팀 버전은 새 파일로 저장한 뒤 커밋합니다.

```
$ git add CODEBASE.md codebase-submissions
$ git commit -m "2팀 PR 병합: CODEBASE.md는 3팀 버전 유지, 2팀 버전은 보존"
$ git log --oneline --graph
*   e9d4481 2팀 PR 병합: CODEBASE.md는 3팀 버전 유지, 2팀 버전은 보존
|\
| * 2d3c517 2팀 코드베이스 설명
* | f605f25 3팀 코드베이스 설명
|/
* 117e959 과제 시작
```

그래프에서 두 갈래(3팀, 2팀)가 병합 커밋 하나로 다시 만나는 것이 보입니다. 병합 커밋은 부모가 둘이라는 점도 확인해 두세요(강의노트 03 4.3절).

실제로 2팀 PR에 남긴 완료 댓글은 다음과 같습니다.

```
2팀의 3주차 실습 PR이 merge되었습니다. data/teams.js에 2팀 정보가
정상 반영되었습니다. 다만 CODEBASE.md는 먼저 merge된 3팀 버전과
경로가 겹쳐 자동 병합이 안 되어, 2팀이 작성하신 버전은 삭제하지 않고
codebase-submissions/2팀-CODEBASE.md에 그대로 보존해두었습니다.
```

주의 · 남의 fork에 push하려면

B 방법은 교수가 학생 fork의 브랜치에 push해야 합니다. GitHub은 PR을 열 때 **[Allow edits by maintainers]**가 켜져 있으면 받는 쪽 저장소 관리자가 PR 브랜치에 push할 수 있게 해 줍니다. 이 설정을 끄면 B는 할 수 없고 A나 C를 써야 합니다. 거꾸로, 이 설정이 켜진 PR은 상대가 내 브랜치를 고칠 수 있다는 뜻이니, 다음 작업을 하기 전에 pull부터 하세요.

제 6 장

기다리지 말고 동시에

6.1 병렬성

병합이 돌아가는 동안 교수는 학생들에게 물었습니다. AI로 일할 때 가장 중요한 것이 무엇일까? 교수의 답은 **병렬성**이었습니다. 한 번에 한 가지 일만 하는 것이 아니라, 여러 일을 동시에 돌릴 수 있어야 합니다.

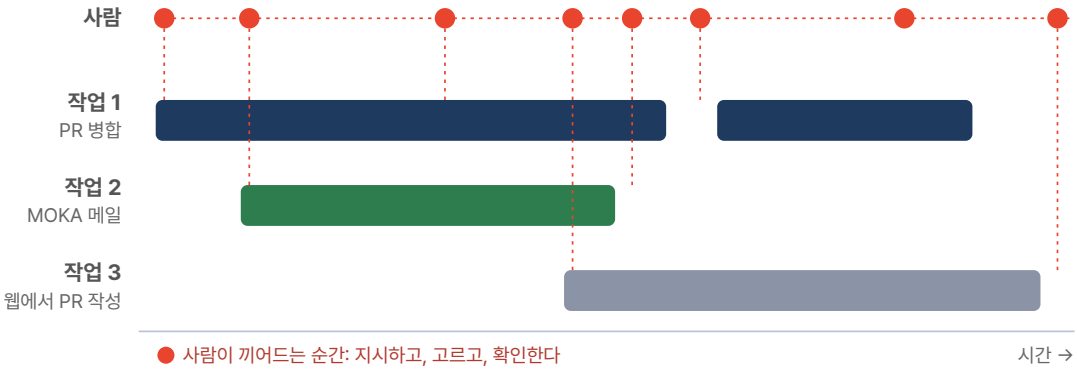


그림 6.1 AI 작업 여러 개를 동시에 돌리고, 사람은 그 사이를 오가며 지시하고 고르고 확인한다

AI가 PR을 병합하는 동안 사람이 화면만 보고 있으면 시간이 아깝습니다. 수업에서 교수는 병합 작업을 돌려 둔 채 다른 창으로 넘어가, MOKA가 학생들에게 메일을 보내는 작업을 이어서 시켰습니다. 요즘 수업 메일은 교수가 결과만 확인하고, 보내는 일은 MOKA가 합니다. 그사이 병합 작업이 끝나면 다시 돌아와 결과를 확인했습니다.

> **수업에서**
>
> 교수는 "한 사람이 적어도 네다섯 개의 일을 동시에 쳐낼 수 있어야 한다"고 했습니다. 일 자체는 이제 AI가 하고, 사람은 그 일을 어떻게 해야 할지만 고민하면 되기 때문입니다. 교수는 아직 인간에게 남은 유일한 장점이 **의지**라고 했습니다. 무엇을 하겠다는 의지를 여러 갈래로 나눠 한 번에 여러 일을 실행시키는 것이 AI 시대의 생산성이라는 이야기입니다. 그러려면 화면(모니터)도 여럿이면 좋습니다.

6.2 결과는 눈으로 확인한다

AI가 병합을 마쳤다고 보고하자, 교수는 저장소 폴더를 VS Code로 열어 결과를 확인했습니다. `CODEBASE.md`, `index.html`, `README.md`, `data/` 폴더와 새로 생긴 `codebase-submissions/` 폴더 안에 팀별 문서가 들어와 있는 것을 보여 주었습니다.

AI의 보고만 믿지 말고, 이렇게 한 번은 결과물을 직접 열어 보세요. 파일 탐색기, VS Code, GitHub 웹 화면 어디서든 됩니다. `index.html`을 브라우저로 열어 네 팀의 카드가 다 나오는지 보는 것도 좋은 확인입니다.

6.3 웹 화면도 AI가 조작한다

병합을 마친 뒤 교수는 각 팀에 완료를 알리는 일을 시켰습니다. 이어서 GitHub 웹 화면에서 해야 하는 일도 AI에게 맡겼습니다. 저장소 설정처럼 웹에서만 바꿀 수 있는 것들이 있고, PR 설명처럼 웹에서 쓰는 편이 자연스러운 것도 있습니다.

> **AI에게 이렇게 시킨다**
>
> > 이건 gh CLI로 하면 돼. 안 되는 건 웹 페이지에서 네가 직접 하나씩 넣어 줘.

이렇게 시키면 AI가 브라우저를 열고 페이지를 직접 조작합니다. 이 기능을 **computer use**라고 부릅니다. 수업 화면에서는 왼쪽에서 AI가 브라우저를 움직이며 입력하고, 교수는 오른쪽에서 다른 일을 했습니다. 교수가 마우스를 움직인 것이 아니라 AI가 한 일입니다.

교수는 이 방식을 한번 해 보라고 권했습니다. 사용자 설정을 바꾸거나 토큰을 발급해 다른 서비스에 넘기는 일처럼, 명령줄로는 안 되고 웹에서 여러 번 클릭해야 하는 일이 생각보다 많기 때문입니다.

> **주의 · 브라우저를 맡길 때**
>
> AI가 조작하는 브라우저에는 여러분의 로그인이 그대로 살아 있습니다. 결제, 삭제, 공개 범위 변경처럼 되돌리기 어려운 버튼은 AI가 누르기 전에 확인을 받도록 하세요. 비밀번호 입력은 AI에게 맡기지 말고 직접 하세요.

제 7 장

마무리: README 갱신과 upstream 동기화

7.1 commit은 저장, push는 원격으로

병합이 모두 끝나자 교수는 마지막으로 이렇게 시켰습니다.

> **AI에게 이렇게 시킨다**
>
> > 현재 버전으로 README 업데이트해 주고, 커밋한 다음 푸시해 줘.

교수가 학생들에게 물었습니다. 커밋은 저장하라는 뜻인데, 푸시는 어디에 하는 거지? 몇 명이 머뭇거렸고, 답은 원격 저장소, 곧 GitHub이었습니다. commit은 내 컴퓨터의 저장소에 기록을 남기는 일이고, push는 그 기록을 원격 저장소로 올리는 일입니다(강의노트 03 5장). push하지 않은 커밋은 내 컴퓨터에만 있습니다.

AI가 푸시했다고 보고했지만, GitHub 화면을 새로 고쳐 봐도 무엇이 바뀌었는지 한눈에 보이지 않았습니다. 교수는 AI에게 무엇을 바꿨는지 물었고, AI는 README 위쪽에 진행 상황(1~4팀 PR 병합 완료)과 새 폴더 설명을 넣었다고 답했습니다. 실제 커밋은 다음과 같습니다.

```
36ce065 README를 현재 저장소 상태로 업데이트
- CODEBASE.md, codebase-submissions/ 구조 설명 추가
- 1~4팀 PR 병합 완료 상태 반영
```

"다 했다"는 보고를 받으면 "무엇을 바꿨어?"라고 되묻는 습관을 들이세요. 커밋 메시지와 diff가 그 답입니다.

7.2 가장 최신 버전은 누가 가지고 있나

교수가 다시 물었습니다. 지금 이 프로젝트의 가장 최신 버전은 누가 가지고 있지? 답은 교수입니다. 네 팀의 작업이 모두 교수 저장소에 합쳐졌으니, 각 팀 저장소는 이제 뒤처져 있습니다. 1팀 저장소에는 2~4팀의 작업이 없습니다.

그럼 여러분은 무엇을 해야 할까요? 학생의 답은 "pull을 해야 한다"였고, 교수는 팀장뿐 아니라 팀원 모두가 해야 한다고 덧붙였습니다. 보통은 팀장이 먼저 교수 저장소를 받아 팀 저장소에 반영하고, 팀원이 팀 저장소를 pull하는 구조입니다. 팀원이 교수 저장소를 바로 pull해도 상관은 없습니다.

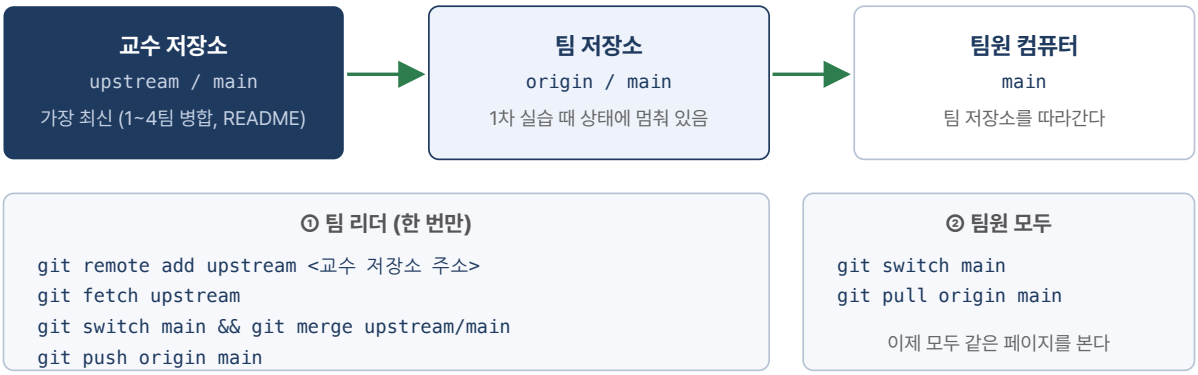


그림 7.1 upstream 동기화. 팀 리더가 교수 저장소를 팀 저장소에 반영하고, 팀원은 팀 저장소를 pull한다

> **핵심 · origin과 upstream**
>
> fork한 저장소에서 일할 때 원격 저장소가 둘 생깁니다. 내 fork(팀 저장소)는 `origin`, 내가 fork해 온 원래 저장소(교수 저장소)는 관례상 `upstream`이라고 부릅니다. 내 작업은 origin으로 push하고, 남들의 최신 작업은 upstream에서 받아 옵니다.

7.3 동기화를 재현해 보면

3팀 리더의 컴퓨터에서 교수 저장소를 upstream으로 등록하고 최신 main을 받아 반영하는 과정입니다. 앞 절의 재현 저장소를 이어서 씁니다.

```
$ git remote add upstream ../prof.git
$ git remote -v
origin        ../team3.git (fetch)
origin        ../team3.git (push)
upstream        ../prof.git (fetch)
upstream        ../prof.git (push)
$ git fetch upstream
From ../prof
 * [new branch]      main       -> upstream/main
$ git merge upstream/main
Updating f605f25..e9d4481
Fast-forward
 codebase-submissions/2팀-CODEBASE.md | 4 ++++
 1 file changed, 4 insertions(+)
 create mode 100644 codebase-submissions/2팀-CODEBASE.md
```

```
$ git push origin main
To ../team3.git
   f605f25..e9d4481  main -> main
```

실제 저장소라면 `../prof.git` 자리에 교수 저장소의 GitHub 주소가 들어갑니다. `git remote -v`에 origin과 upstream이 나란히 보이고, `fetch`로 받아 온 교수 저장소의 main은 `upstream/main`이라는 이름으로 들어옵니다. 3팀 저장소는 교수 저장소보다 뒤에 있을 뿐 새 커밋이 없으니 `Fast-forward`로 따라잡았습니다.

7.4 2차 실습

교수는 이 내용을 README에 2차 실습 안내로 넣으라고 AI에게 시켰습니다. 실제로 추가된 안내의 뼈대는 다음과 같습니다.

1. **팀 리더:** 팀 저장소에 교수 저장소를 `upstream`으로 추가하고, 최신 main을 받아 팀 저장소 main에 반영해 push합니다. 이미 upstream이 있으면 `git remote -v`로 확인하고 추가는 건너뜁니다.
2. **팀원:** 자기 컴퓨터의 main을 팀 저장소 main과 맞춥니다(`git pull origin main`). 이제 모든 팀원이 교수와 같은 페이지를 봅니다.
3. **다시 한 바퀴:** 1차와 같은 흐름(브랜치 → 팀 저장소로 PR → 리뷰 댓글 → 병합 → 교수 저장소로 PR)을 한 번 더 연습합니다. 병합되면 1차 때처럼 PR에 완료 댓글이 달립니다.

> **AI에게 이렇게 시킨다 · 2차 실습**
>
> > 이 저장소는 교수 저장소를 fork한 거야. 교수 저장소를 upstream으로 등록하고, 최신 main을 받아서 우리 main에 반영한 다음 push해 줘. 무엇이 바뀌었는지도 알려 줘.

GitHub 웹에서는 fork 저장소 첫 화면의 **[Sync fork]** 버튼으로도 같은 일을 할 수 있고, `gh`로는 `gh repo sync`가 같은 일을 합니다. 어느 방법이든 결과는 같습니다. 팀 저장소의 main이 교수 저장소의 main을 따라잡습니다.

> **주의 · 동기화 전에 내 작업부터**
>
> 팀 저장소나 내 컴퓨터에 아직 올리지 않은 작업이 있으면, 먼저 commit해 두고 동기화하세요. 교수 저장소와 같은 파일의 같은 줄을 고쳤다면 이번에는 여러분 쪽에서 충돌이 납니다. 그때는 5장에서 배운 대로 AI에게 선택지를 달라고 하세요.

부록 A

수업에서 쓴 요청 모음

수업에서 Claude Code에 한 말을 순서대로 모았습니다. 녹화에서 옮기며 조금 다듬었습니다. 여러분 저장소 이름에 맞게 바꿔 쓰세요.

단계	요청
가져오기	GitHub에 있는 swe-2026-git-practice 저장소를 가져올 거야. 현재 폴더 기준으로 어디에 두면 좋을지 생각해 보자.
가져오기	Lectures 폴더를 만들고 거기에 클론하자.
병합 계획	포크해서 PR한 걸 gh를 사용해서 머지할 거야. 어떻게 할지 고민해 보자.
설치	(오류를 붙여 넣고) 이렇게 나와. / 새 터미널 열었는데도 안 돼.
확인	내가 로그인했어. 네가 직접 확인해 줘.
충돌	충돌의 문제점들을 코드 단위로 나에게 설명해 주고, 내가 의사결정할 수 있게 해 줘.
알림	각 팀에 병합이 완료됐다고 알려 줘.
웹	gh CLI로 되는 건 CLI로 하고, 안 되는 건 웹 페이지에서 네가 직접 하나씩 넣어 줘.
마무리	현재 버전으로 README 업데이트해 주고, 커밋한 다음 푸시해 줘. / 뭘 바꾼 거야?
2차 안내	1차 실습은 끝났다고 README에 적고, 모든 팀원이 교수 저장소를 pull해서 같은 페이지를 만든 다음 다시 PR하도록 README를 업데이트해 줘.

부록 B

gh 명령 한눈에

외울 필요는 없습니다. AI의 작업 기록에 이런 줄이 지나가면 무슨 일인지 알아보는 표로 쓰세요.

분류	명령	하는 일
로그인	`gh auth login`	브라우저로 GitHub 계정에 로그인
로그인	`gh auth status`	로그인 상태 확인
저장소	`gh repo clone <주인/이름>`	저장소 가져오기 (`git clone`과 같음)
저장소	`gh repo fork <주인/이름>`	내 계정으로 fork
저장소	`gh repo sync <내 fork>`	fork를 원래 저장소와 동기화
PR	`gh pr list`	열린 PR 목록
PR	`gh pr view <번호>`	PR 설명과 병합 가능 여부
PR	`gh pr diff <번호>`	PR이 바꾼 내용
PR	`gh pr checks <번호>`	자동 검사 결과
PR	`gh pr checkout <번호>`	PR 브랜치를 내 컴퓨터로 받아 옮기기
PR	`gh pr create`	PR 열기
PR	`gh pr merge <번호> --merge`	PR 병합 (병합 커밋 만들기)
PR	`gh pr comment <번호> -b "..."`	PR에 댓글 달기

부록 C

헷갈리는 짝

짝	앞의 것	뒤의 것
git · gh	저장소 안의 기록을 다루는 도구	GitHub 서비스의 일(PR, 이슈, fork)을 다루는 도구
CLI · GUI	글자로 명령을 주고받는 화면	버튼과 창으로 조작하는 화면
origin · upstream	내 fork(팀 저장소)	fork해 온 원래 저장소(교수 저장소)
commit · push	내 컴퓨터의 저장소에 기록	원격 저장소(GitHub)로 올리기
clone · fork	원격 저장소를 내 컴퓨터로 복사 (Git)	저장소를 내 GitHub 계정으로 복사 (GitHub)
일회용 코드 · 토큰	로그인할 때 한 번 쓰고 버리는 코드	로그인 뒤 계속 쓰는 출입증. 비밀번호처럼 지킨다
자동 병합 · 충돌	서로 다른 곳을 고쳐서 Git이 합침	같은 곳을 다르게 고쳐서 사람이 정해야 함

부록 D

복습 문제

답을 외워 쓰기보다 친구에게 말로 설명한다고 생각하고 풀어 보세요.

1. "포크해서 PR한 걸 gh를 사용해서 머지하자"라는 문장을 fork, PR, gh, merge의 뜻을 넣어 풀어 쓰시오.
2. CLI가 무엇의 약자이고 무엇을 뜻하는지 쓰고, AI 에이전트가 GUI보다 CLI를 다루기 쉬운 이유를 설명하시오.
3. git과 gh는 각각 무엇을 다루는 도구인가? PR을 다루려면 왜 gh가 필요한가?
4. 프로그램을 설치한 직후 터미널이 그 명령을 찾지 못하는 이유와 해결 방법을 쓰시오.
5. `gh auth login`은 왜 해야 하는가? 이때 오가는 일회용 코드와 토큰은 어떻게 다른가?
6. 3주차 실습에서 3팀의 PR은 바로 병합됐는데 나머지 세 팀의 PR은 충돌한 이유를 설명하시오. 같은 과제에서 `data/teams.js`는 충돌하지 않은 이유도 함께 쓰시오.
7. 충돌 표시(`<<<<<<<`, `=======`, `>>>>>>>`)는 각각 무엇을 나누는가?
8. AI에게 "충돌을 해결해 줘" 대신 "내가 의사결정할 수 있게 해 줘"라고 시키는 이유를 수업 내용을 바탕으로 설명하시오.
9. 충돌을 푸는 네 가지 방법(A~D) 가운데 하나를 골라 장단점을 쓰시오. 수업에서 B를 쓸 수 있었던 조건은 무엇인가?
10. commit과 push의 차이를 쓰시오. push하지 않은 커밋은 어디에 있는가?
11. fork한 저장소에서 origin과 upstream은 각각 무엇을 가리키는가? 2차 실습에서 팀 리더와 팀원이 할 일을 순서대로 쓰시오.
12. AI로 일할 때 병렬성이 왜 중요한지, 그리고 AI가 "다 했다"고 보고했을 때 사람이 해야 할 일을 설명하시오.

— 끝 —